⚠ **TRAINING EXAMPLE — NOT A REAL IEC 62304 DELIVERABLE**

This project is a **training site, not a real medical device** under any regulatory definition — it is not Software as a Medical Device (SaMD) or Software in a Medical Device (SiMD). This page is not a genuine IEC 62304 deliverable: it illustrates the *kind* of document a real SaMD/SiMD project would produce. See the document register for the full explanation.

Status: Documented — this project's other strong piece of evidence. Clause: 9 — Problem Resolution Process · **Register:** Document Register

What the standard requires (Class C — every sub-clause of Clause 9)

REF	TITLE	APPLIES AT	OUTPUT
9.1	Prepare problem reports	A, B, C	A problem report per problem detected, with a criticality statement and information aiding resolution
9.2	Investigate the problem	A, B, C	Documented investigation outcome, and either a change request or a documented rationale for no action
9.3	Advise relevant parties	A, B, C	—
9.4	Use change control process	A, B, C	—
9.5	Maintain records	A, B, C	Records of problem reports and their resolution, including verification, and an updated risk management file
9.6	Analyse problems for trends	A, B, C	—

REF	TITLE	APPLIES AT	OUTPUT
9.7	Verify software problem resolution	A, B, C	Verification the resolution closed the problem, reversed adverse trends, was implemented correctly, and introduced no new problems
9.8	Test documentation contents	A, B, C	Results, anomalies, software version, hardware/software test configuration, test tools, date, tester

This process is implemented in code, not only described

Unlike most documents in this set, Clause 9 has a direct, working implementation: `tests/anomaly_log.csv`, produced by `load_anomaly_log()` and `reconcile_anomaly_log()` in `tests/test_site.py`. What follows maps each sub-clause to what that code actually does — including where it falls short of the full requirement.

9.1 — Problem reports

Every check that fails in a test run becomes a row, keyed by *(test group, check label)* — see the site's own Anomaly log section for the full mechanics. Each row carries an id (`ANOM-####`), the failing detail captured at the moment of failure, and how many consecutive runs it has failed (`times_seen`).

9.1 also asks for a criticality statement — now partly present. The anomaly log's fields are `id, status, group, test, first_seen, last_seen, times_seen, closed_on, risk_of_harm, detail` . `risk_of_harm` is a binary, not the graded severity scale 9.1 more fully implies: Yes if the failing check is tagged `risk=True` — under this project's reflexive ISO 14971 hazard model, a check whose failure means a reader could form an incorrect belief about what IEC 62304 requires (see 11 — Risk Management File). Everything else is still treated as equally urgent by the tooling; a human reading the log supplies the finer-grained judgement of "how bad is this" that 9.1 expects the report itself to carry. Recorded as a partial gap below.

9.2 — Investigation

The `detail` field is the investigation's evidence — the actual value the check received versus what it expected, e.g. `['planning', 'planning']` for a duplicate-id defect. What is not automated: 9.2's requirement for **either a change request or a documented rationale for taking no action**. The current process closes an anomaly only when the underlying check starts passing again — there is no path for "investigated, judged not a real problem, closing with rationale but without a code change," which a stricter reading of 9.2 would want to support (e.g. a flaky check, or a requirement that turned out to be wrong rather than the software).

9.3 — Advising relevant parties

The console summary (`ANOMALY LOG` block printed at the end of every run) is the notification mechanism, read by whoever ran the suite. There is no broader distribution — no issue tracker integration, no notification to anyone beyond the person at the keyboard — appropriate for a solo-maintained project, a real gap for a team of any size.

9.4 — Change control

Handled by 12 — Configuration Management Plan: a fix is a git commit, reviewed the same way any other change is.

9.5 — Records, including an updated risk management file

`tests/anomaly_log.csv` **is** the problem-report-and-resolution record — committed to the repository, so its history is retrievable the way 8.3 asks of any configuration item (see 12). What 9.5 also asks for — an **updated risk management file** — is now linked automatically for anomalies the risk model actually covers: `escalate_risk_anomalies()` rewrites the "Escalations from the problem log" section of 11 on every run, listing every currently-open `risk_of_harm=Yes` anomaly by id. What is still manual is upstream of that: 11's own traceability table (the hazard → cause → risk control → verification chain for the *known* risks) is not regenerated from the anomaly log and still needs a

maintainer to update it by hand when a new kind of hazard is found — the escalation section surfaces that a risk-tagged check is failing, it does not write the analysis of why.

9.6 — Trend analysis

`times_seen` is a rudimentary trend signal — an anomaly open for eight consecutive runs is visibly different from one open for one — but there is no analysis *across* anomalies (e.g. "three of this month's anomalies were async-error-handling regressions," which might point at a systemic weakness rather than three unrelated bugs). This is exactly the kind of pattern 9.6 asks a maintainer to look for, and currently nothing in the tooling surfaces it; it would have to be read by eye from the CSV.

9.7 — Verifying the resolution

This is the strongest-covered sub-clause of the group. Because `reconcile_anomaly_log()` re-derives status from the **current full test run** rather than from a person's say-so, closing `ANOM-0001` requires:

- the original check to actually pass again (closes the problem);
- every *other* check in the group that ran to still be evaluated in the same pass, so a fix that broke something else shows up as a **new** anomaly in the same report rather than passing silently (covers "introduced no new problems");
- the anomaly's `(group, test)` key to be exactly the one that was failing (covers "implemented in the right software and activities" at the granularity this project tracks).

What is *not* covered: "reversed adverse trends" in the 9.6 sense — a problem can close and reopen indefinitely (verified directly — see the site's own Anomaly log section's worked example of `ANOM-0001` closing then reopening) without that oscillation itself being flagged as a trend worth investigating.

9.8 — Test documentation contents

Identical evidence to 08 — System Testing §5.7.5 — the same gaps apply here: no software-version stamp, no date stamp beyond the anomaly log's own `first_seen` / `last_seen` columns.

Gaps

- **Criticality/severity is now recorded, but only as a binary** (9.1) — a `risk_of_harm` column (Yes/No), set from whether the failing check is tagged `risk=True` in `tests/test_site.py`, not the graded severity scale (e.g. critical/major/minor) 9.1 more fully implies. See 11 — Risk Management File.
- **No "investigated, no action needed" closure path** distinct from "the check passes again" (9.2).
- **No cross-anomaly trend analysis** (9.6) — data exists (`times_seen`, dates) but nothing currently aggregates it into a signal.
- **The risk management file now updates itself automatically** (9.5) — no longer a gap in the case it originally named: `reconcile_anomaly_log()` escalates every open `risk_of_harm=Yes` anomaly into 11 by id, on every run. What remains manual is upstream of that: deciding whether a *given* check's failure belongs in the hazard model in the first place is a judgement made once, when the check is written, not something the tooling infers — a check that is actually safety-relevant but was never tagged `risk=True` would still not escalate.

Three of these four are no longer silently missing; the fourth (9.6) still is — this document is where all four are recorded either way, per the very process it describes.